ISE-supported erasure of residual shares

Hao Cheng[1,4,5], Linus Mainka[2], Daniel Page[3] and Kostas Papagiannopoulos[2]

[1] School of Cyber Science and Technology, Shandong University, Qingdao, China.
hao.cheng@sdu.edu.cn
[2] Informatics Institute, University of Amsterdam, Amsterdam, The Netherlands.
{l.mainka,k.papagiannopoulos}@uva.nl
[3] School of Computer Science, University of Bristol, Bristol, UK.
daniel.page@bristol.ac.uk
[4] State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China.
[5] Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Qingdao, China.

Abstract. Careful management of shares stored in the architectural, general-purpose register file is an important aspect of software-based implementations of masking, because it can impact the associated side-channel leakage. The erasure of residual shares, i.e., shares whose useful lifetime has expired, is one component of such management: erasing them can, for example, prevent subsequent, unintended share recombination. To support effective share erasure, this work makes two contributions. First, we present an analysis of the underlying problem, systematising associated concepts and terminology, and offering a concrete, motivating example. Second, we present the design, implementation, and evaluation of two components which can support solutions of said problem. These are 1) a *policy*, namely an extended ABI which provides clear semantics regarding the responsibility of caller and/or callee functions to erase shares, and 2) a *mechanism* for realising said policy, namely an extended ISA (or ISE) which allows more efficient erasure of shares than via the ISA alone. Although generic in nature, we present both components using the RISC-V base ISA; an associated prototype ISE implementation uses Ibex as a base core. Our evaluation results confirm that combined use of the components can eliminate leakage related to residual shares both effectively and efficiently.

Keywords: side-channel analysis, masking, RISC-V, ISE

1 Introduction

The usage of modern embedded computing devices in security-critical applications demands attention to a broad, diverse, and challenging attack landscape. The general concept of an implementation attack, wherein the attacker can passively observe or actively influence the behaviour of a target device respectively, is one example; more specific instances of passive, i.e., side-channel attacks include Differential Power Analysis (DPA) [KJJ99] and variants thereof.

Masking in theory. Countermeasures against side-channel attacks are often classified as being based on hiding or masking [MOP07, Chapters 7, 9]. Focusing on the latter, a d-th order Boolean masking scheme can be viewed as harnessing the concept of secret sharing [Sha79] by representing a variable x as $\hat{x} = \langle \hat{x}[0], \hat{x}[1], \ldots, \hat{x}[d] \rangle$, i.e., as $d+1$ statistically independent shares, where $x = \hat{x}[0] \oplus \hat{x}[1] \oplus \cdots \oplus \hat{x}[d]$. Some function f which

would be applied to x is translated into a compatible, masked alternative function $\hat{f}$ applied to $\hat{x}$.

It is common to describe an implementation of a large masked function $\hat{f}$ in terms of n smaller, separate masked functions $\hat{f}_i$ for $0 \leq i < n$. Features such as modularity and composability motivate this approach, in terms of both the implementation and security proofs for it. For example, one might use fine-grained masked functions, or gadgets, such as SecAnd ("secure, masked AND") and SecOr ("secure, masked OR") by Biryukov et al. [BDLU17, Table 1] to implement a masked adder function used by an Add-Rotate-XOR (ARX) cipher like Speck [DGL17]. Similarly, one might use coarse-grained masked functions such as `SubBytes`, `MixColumns`, and `Remask` to construct a masked AES-128 implementation [MOP07, Pages 228–231].

Masking in practice. The masking scheme used by $\hat{f}$ will often enjoy theoretical guarantees of security and composability. The translation of such guarantees into practice will, however, depend on additional assumptions such as the following.

Assumption 1. *The leakage observed in a practical implementation matches the model used in theory. That is, a device executing an implementation of $\hat{f}$ should leak information in a manner sufficiently close to the leakage model used by the security proof of $\hat{f}$.*

For example, one might verify that the 2-share Speck implementation in [DGL17] does not directly compute $\hat{x}[0] \oplus \hat{x}[1]$, i.e., the shares are not directly recombined into x causing leakage. However, there is now significant evidence [BWG+22] that a concrete micro-architecture may indirectly recombine shares while executing instructions and yield the same leakage. If such detail is omitted from the model used in the security proofs, then Assumption 1 is violated and the implementation is insecure. The underlying issue can be framed in relation to the Instruction Set Architecture (ISA), which acts as an interface between software (i.e., instructions) and hardware (i.e., a micro-architecture executing them).

Assumption 2. *The practical implementations of $\hat{f}_i$ and $\hat{f}_j$, with $i \neq j$, do not invalidate theoretical guarantees about their composability. That is, an implementation of $\hat{f}_i$ can be freely composed with $\hat{f}_j$ without causing leakage not catered for by the security proof.*

This assumption implies an associated challenge, namely how to translate an abstract notion of composability (i.e., defined in terms of algorithmic gadgets) into a concrete notion (i.e., defined in terms of instructions and functions that implement said gadgets, and their execution). Specifically, gadgets could be provably composable in a theoretical specification yet fail to compose when instantiated in a practical implementation. For example, the 2-share Speck implementation in [DGL17] could use trivially composable Probe Isolating Non-Interference (PINI) gadgets [CS20]. However, it is possible (as we show in Section 3.4) that share recombination occurs as the result of security-agnostic register allocation and (re)use. For example, a callee function may overwrite share-containing registers, previously placed there by a caller function, violating Assumption 2. The underlying issue can be framed in relation to the Application Binary Interface (ABI), which acts as an interface between software. The ABI defines the interface between caller and callee functions via a function call convention, which is supported by or includes a register convention, i.e., a set of roles for distinguished registers.

Contributions. This paper focuses on one aspect of the model and composability challenges outlined above, namely the secure and efficient erasure of residual shares. Deferring a complete explanation to Section 2, this refers to the idea that shares whose useful lifetime has expired should be erased (or zeroised) so as to avoid subsequent, unintentional recombinations. More concretely, we focus on a general-purpose, software-based implementation of masked functions. Given that a callee function $\hat{f}_i$ can be called by (and thus arbitrarily

composed with) some (possibly unknown) caller function $\hat{f}_j$, the goal is to efficiently minimise the potential for practical model and composability failures (i.e., violations of Assumptions 1 and 2). In support of this goal, we make the following contributions:

1. We present an analysis of the underlying problem, systematising associated concepts and terminology, and offering a concrete, motivating example; this example is based on use of Rosita [SSB+21], and highlights the impact of unintentional share recombination stemming from ineffective management of residual shares.
2. We present the design, implementation, and evaluation of two simple components which can be used in combination to support solutions of said problem. First, a share erasure *policy*: we define an extended ABI, which provides clear semantics regarding responsibility of either caller and/or callee function to erase shares. Second, a share erasure *mechanism* for realising said policy: we define an extended ISA, i.e., an Instruction Set Extension (ISE), which allows more efficient erasure of shares than via the ISA alone. We claim that, in combination, use of these components can facilitate implementations which are 1) more effective, in the sense that the potential for practical model and composability failures due to automation is minimised, and/or 2) more efficient, in the sense that the overhead of any share erasure required to do so is minimised.

Scope and limitations. We (deliberately) consider a scope which is limited in the following ways. First, our scope is limited to one, specific architectural resource, i.e., the register file; this means other, e.g., micro-architectural (e.g., pipeline registers) and uncore resources are out of scope. For example, we exclude consideration of memory (meaning, e.g., shares stored on the stack). Our rationale is that erasure of shares stored in memory would have to be performed via the associated interface, unless invasive alteration of said interface and/or the memory is also considered. Having ruled out doing so (in part because it may be impossible for external, third-party memory IP), it is difficult to see any opportunity to improve the obvious ISA-based approach, i.e., overwriting each residual share with zero using a store instruction. Second, our scope is limited to one, specific leakage effect, i.e., overwriting in said register file; this means alternatives, e.g., the neighbour effect, are out of scope. Third, our scope is limited to support for implementation, not identification of residual share erasure. For example, we focus on provision of instructions to erase residual shares; we do *not* focus on identification of those residual shares, instead deferring this challenge to the scVerif tool of Barthe et al. [BGG+21], which offers a well-justified, consistent rationale for which residual shares to erase. Fourth, our scope is limited to manual (i.e., by a developer) not automatic (e.g., by a compiler) implementation of residual share erasure. However, we claim both components are orthogonal to but are well suited to (semi-)automatic use, e.g., in support of compiler-based identification and instrumentation of share erasure: doing so seems feasible in tools such as Jasmin [ABB+17] (cf. Olmos et al. [OBG+24]) or PoMMES [ZM24], for example.

To summarise, and framed against the goal of developing a leakage-free masked implementation, our contributions are therefore not themselves sufficient: they *do not* address *every* relevant challenge. They *do* address *a* necessary challenge, however, and so remain useful in combination with orthogonal approaches (e.g., the fence instruction of Gao et al. [GMPP20], addressing *micro*-architectural resources) and within a wider methodology (e.g., that of Belaïd at al. [BCM+25]).

2 Background

2.1 Notation

Let $x_{(b)}$ denote x expressed in radix- or base-b. If the base is omitted, we use decimal base, i.e., $b = 10$. Let $x \leftarrow y$ denote assignment of y to x. Let $\neg$, $\wedge$, $\vee$, and $\oplus$ denote the Boolean NOT, AND, (inclusive) OR, and (exclusive OR, or) XOR operators respectively, and $x \ll y$ and $x \lll y$ (resp. $x \gg y$ and $x \ggg y$) denote left-shift and left-rotate (resp. right-shift and right-rotate) of x by y bits. Let $\mathrm{HW}(x)$ and $\mathrm{HD}(x, y)$ respectively denote the Hamming weight of x and Hamming distance between x and y. Let $x = \mathbf{1}^w$ and $y = \mathbf{0}^w$ denote w-bit words such that $\mathrm{HW}(x) = w$ and $\mathrm{HW}(y) = 0$ respectively, i.e., $x_i = 1$ and $y_i = 0$ for $0 \leq i < w$.

Let $\mathsf{MEM}[i]^b$ denote a b-byte access to some byte-addressable memory, using the effective address i; when $b = 1$, we omit the access granularity. Let $\mathsf{GPR}[i]$, for $0 \leq i < e$, denote the i-th, w-bit entry in an e-entry general-purpose register file.

2.2 Erasure of data

Per evidence such as the Common Weakness Enumeration (CWE) [CWE], the challenge of effective data sanitisation, i.e., erasure of *security-critical* data from some data storage resource in the memory hierarchy to prevent subsequent access, forms a central principle in security-related development. A specified policy for data sanitisation will involve one or more mechanisms for data erasure; in some cases that mechanism may be realised in (or supported by) hardware, but in other cases it must be realised in software alone. Where the data relates to a cryptographic use-case (see, e.g., FIPS 140-3 [FIP19, Section 2]), the term zeroisation[1] is used synonymously. Using the terminology by Chow et al. [CPGR05], the goal of zeroisation is to control the lifetime of, e.g., key material within the memory hierarchy, and thereby limit exposure. Put simply, one explicitly overwrites the security-critical data with some security-neutral data (such as, but not limited to zero) once the useful lifetime expires. This prevents subsequent accesses, and thus certain attack vectors (see, e.g., [HSH+09]) whose aim would be to recover the security-critical data.

However, realising data erasure effectively can be challenging. A common issue is the abstraction of, and thus a lack of direct control over the data storage resource, for which Olmos et al. [OBG+24] consider two specific root causes. First, the physical properties of the resource may not be exposed to the user, a fact that Gutmann [Gut96] showcases in non-volatile, magnetic storage media, where writing to the resource can result in persistent (and recoverable) residual data, even *after* having been overwritten. The second cause stems from fact that the use of the resource may be (semi-)automated, e.g., by a compiler on behalf of a developer. Simon et al. [SCA18] present an exploration of this, with [SCA18, Section 2.2.2] focusing on "*erasure of sensitive data such as crypto keys from RAM*". The central observation is that aggressive, or at least security-agnostic optimisation (e.g., dead code elimination) may remove instructions inserted to realise data erasure.

2.3 Erasure of shares

Each share $\hat{x}[i]$ within a masked software implementation is clearly security-critical, in a manner akin to key material, and thus demands explicit management to, e.g., prevent recovery by an attacker. For example, a fundamental requirement is that shares are never[2] recombined: doing so would unmask, and thereby produce leaking relating to the

[1] A large number of roughly synonymous terms exist: depending on the context, one might see "scrub", "clear", "wipe", or "purge" used, for example.

[2] One caveat would be so-called "safe" recombinations, such as unmasking of ciphertext shares produced by a (masked) encryption implementation.

underlying data. One can formally guarantee this requirement is satisfied by an abstract specification of $\hat{f}$ (see, e.g., Prouff and Rivain [PR13]). However, Papagiannopoulos and Veshchikov [PV17] and Marshall et al. [MPW22], among others, systematically explored cases where executing the concrete implementation of $\hat{f}$ may unintentionally violate this requirement. To avoid such cases, a range of associated leakage effects such as

Definition 1 (Overwrite effect). *The overwrite effect [PV17, Section 3.1] occurs when a share $\hat{x}[i]$ is overwritten by another share $\hat{x}[j]$, where $0 \leq i, j \leq d$ and $i \neq j$; this means the shares interact, implying share recombination and associated leakage.*

must be considered and addressed. For example, consider the following:

Definition 2 (Residual share). *A residual share is any share stored in a register* GPR[i] *whose lifetime has expired.*

As one component of a wider methodology (i.e., doing so will not address leakage stemming from *non*-residual shares, nor other leakage effects), timely erasure of residual shares from the register file can help prevent unintentional share recombination, and hence leakage due to the overwrite effect. Set against the goal of developing a masked RECTANGLE [ZBL+15] implementation, Papagiannopoulos and Veshchikov [PV17, Section 5] consider doing so. From their description one can infer the following workflow:

Definition 3 (Share erasure workflow). *A workflow for management of residual shares involves four steps, i.e., 1) accept an input program, 2) identify residual shares, 3) driven by some policy, use some mechanism (e.g., insert additional instructions) to erase them, and thereby 4) produce an output program which is functionally equivalent but ideally more secure, i.e., hardened such that leakage stemming from residual shares is eliminated.*

2.4 scVerif

To address the challenge of "*building efficient, provably secure, and practically hardened implementations of masked algorithms*", Barthe et al. [BGG+21] present a Domain Specific Language (DSL) called IL which allows expression of two device-dependent components: 1) a model of instructions and their functional semantics [BGG+21, Section 2.1], i.e., the ISA supported by the device, and 2) a model of the leakage caused by each instruction [BGG+21, Section 2.2]. The paper focuses on an instruction model based on ARMv6-M and a leakage model based on a Cortex M0+ micro-controller, utilising the `leak` statement to model standard leakage such as value-based computation, as well as recombination effects such as the overwrite effect.

An assembly language implementation can be represented by mapping each instruction to a tuple which includes 1) the instruction address plus 2) a macro invoking the associated instruction semantics. Given an instruction model and a leakage model for some device, plus annotation to specify, e.g., input and output, variable sensitivity and initialisation, and memory content, the associated scVerif[3] tool can formally reason about leakage from such an implementation. At the time of writing it supports the notions of (stateful) t-Non-Interference (t-NI) and (stateful) t-Strong-Non-Interference (t-SNI), allowing verification of the presence or absence of leakage.

Per [BGG+21, Section 4], the evaluation develops and proves the security of several "hardened" gadgets. In doing so, two macros are used to abstractly model the mitigation of specific leakage effects: they respectively offer "*"scrubbing" countermeasures, which purge architecture state*", and "*"clearing" countermeasures, which remove values residing in leakage state*". For example, one might use Scrub(R0) to erase a residual share in GPR[0]; doing so might eliminate associated leakage, and thus allow verification of an implementation fulfilling a given notion of security to succeed. This is particularly relevant,

[3] https://github.com/scverif

Table 1: A summary of the RISC-V register file.

Register			Mnemonic	Purpose	Semantics
GPR[0]	≡	x0	zero	zero	immutable
GPR[1]	≡	x1	ra	return address	caller-save
GPR[2]	≡	x2	sp	stack pointer	callee-save
GPR[3]	≡	x3	gp	global pointer	unallocatable
GPR[4]	≡	x4	tp	thread pointer	unallocatable
GPR[5 − 7]	≡	x5 - x7	t0 - t2	temporary	caller-save
GPR[8 − 9]	≡	x8 - x9	s0 - s1	saved	callee-save
GPR[10 − 11]	≡	x10 - x11	a0 - a1	argument / return value	caller-save
GPR[12 − 17]	≡	x12 - x17	a2 - a7	argument	caller-save
GPR[18 − 27]	≡	x18 - x27	s2 - s11	saved	callee-save
GPR[28 − 31]	≡	x28 - x31	t3 - t6	temporary	caller-save
ISA			ABI		

because efficient, concrete realisation of the SCRUB macro (or sequences thereof) offers a concise problem statement for our work.

2.5 RISC-V

Although Section 3 formulates the residual share and related lifetime concepts using an abstract ISA (with Section 3.4 then presenting a concrete example necessarily based on ARMv6-M), our contributions from Section 4 onward focus exclusively on RISC-V (see, e.g., [Wat16]) as a concrete base ISA. This choice is motivated by the fact that RISC-V makes it (relatively) easy to explore research ideas which involve hardware and software dimensions. First, it adopts strongly RISC-oriented design principles but is highly modular: a sparse, general-purpose base ISA, e.g., RV32I [RV:19, Chapter 2] or RV64I [RV:19, Chapter 5], can be augmented with special-purpose standard and non-standard extensions. Second, the more open nature of the ISA has (arguably) led to a more accessible infrastructure to use as a starting point for further development: open-source base cores and development tool-chains are readily available.

We focus, without loss of generality, on RV32I, i.e., the 32-bit integer RISC-V base ISA, using a $w = \text{XLEN} = 32$ bit word size and an $e = 32$ element register file. Table 1 summarizes the RISC-V register file: this includes the register convention, which supports the associated function call convention [RV:23, Chapters 1,2]. Note that GPR[0] ≡ x0 ≡ zero is fixed to 0 (in the sense that reads from it always yield 0 and writes to it are ignored).

Definition 4 (Register set). *We use the term register set to refer to some set $\mathcal{R}$ where each i-th element $0 \leq \mathcal{R}_i < 32$ is the address (or index) of a general-purpose register listed in Table 1; we use the terms register and register address somewhat synonymously, e.g., saying "a register is in $\mathcal{R}$" if the address of said register is an element of $\mathcal{R}$.*

3 Analysis

This Section attempts to 1) clarify the underlying residual share problem via a conceptual framework and terminology (Sections 3.1, 3.2, 3.3), and 2) offer a concrete, motivating example (Section 3.4) for practical model and composability failures. We assume, without loss of generality, that the context is some masked function implementation $\hat{f}$ of a block cipher which encrypts a plaintext $\hat{m}$ under a cipher key $\hat{k}$ to yield a ciphertext $\hat{c} = \hat{f}(\hat{k}, \hat{m})$. We assume that the developer of *and* gadgets used in said implementation, i.e., each $\hat{f}_i$, are "rational" in the sense that they do not intentionally recombine shares.

3.1 Share lifetime framework

Motivated by the approach of Chow et al. [CPGR05], we provide a simple conceptual framework for understanding the lifetime of a share stored in register GPR[i] during the execution of a masked function $\hat{f}$, as we demonstrate in Figure 1a. We stress that the framework describes residual leakage effects on a low, scheme-agnostic level, utilizing registers and generic assembly instructions.

The lifetime of a share begins with the first instruction (typically memory-related) that writes the share to GPR[i]. Before the first instruction, we assume that the contents of GPR[i] are indeterminate. During and after the share's lifetime, the register can be read from, written to or get indirectly affected by various *safe* instructions that do not cause recombinations. Similarly, during and after the intended lifetime of a share, *hazardous* instructions can reduce the security guarantee. The following definitions specify the particular time periods that are of interest for share erasure.

Definition 5 (Recombination time). *Recombination time is the period of time from the first instruction involving the share in* GPR[i]*, until the first hazard on the share occurs.*

Note that the hazardous instruction may not necessarily read from or write to register GPR[i] directly, e.g., the security guarantee can be reduced by direct register overwrite effects, as well as indirect neighbor or pipeline effects. In other words, depending on the recombination effect, the contents of GPR[i] may or may not be altered.

Definition 6 (Lifetime). *Lifetime is the period of time from the first instruction involving the share in* GPR[i] *until the last instruction intentionally involving the share.*

After the lifetime of a share stored in GPR[i] has expired (i.e, after the last intentional use), we refer to it as *residual share* (Section 2.3, Definition 2). While the share remains useful, implementations would typically strive to maximise the register utilisation (and thus its lifetime) by keeping it in the register file as long as possible and avoiding unnecessary memory load/store overheads that decrease performance.

Definition 7 (Critical region). *Critical region is the period of time from the last intentional use of the share in* GPR[i] *until the first hazard on the share occurs.*

The critical region delimits the time window during which the developer should erase GPR[i] to avoid recombinations from residual shares. Note that recombinations can also be avoided by reordering instructions and changing register operands or by (intentionally or unintentionally) overwriting GPR[i] before the hazard occurs. Still, being able to robustly and efficiently erase register contents remains advantageous to the developer and provides a generic way to deal with hazards that does not presuppose extensive code modifications.

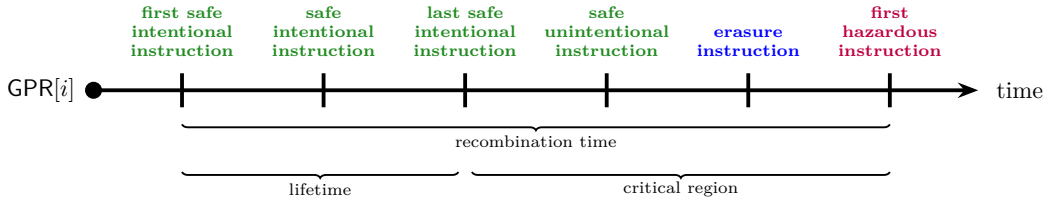
3.2 Intra- and Inter-function leakage of residual shares

Figure 1b demonstrates that residual share recombinations can occur *during* the execution of a masked function, as well as *after* that function has concluded. We distinguish between these two cases with the following definitions.

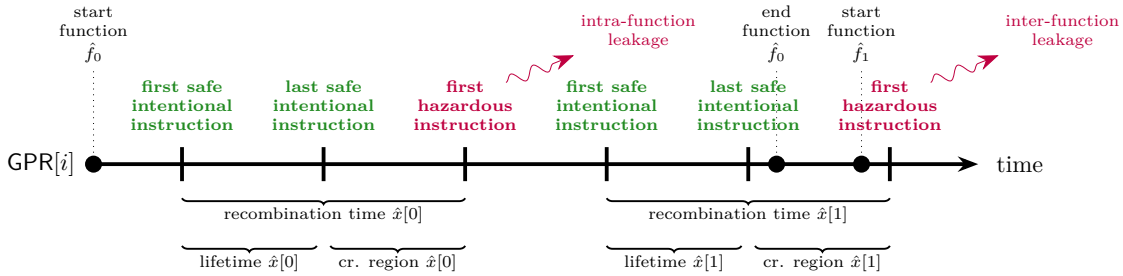
Definition 8 (Intra-function leakage). *Intra-function is any leakage caused by the recombination of residual shares from function $\hat{f}$, during the computation of function $\hat{f}$.*

Assuming a provably secure $\hat{f}$, intra-function leakage is typically caused by implementing $\hat{f}$ on a device whose actual leakage is different than the model used by the security proof (violation of Assumption 1, Section 1). Note that a security proof (if possible) for function $\hat{f}$ under the appropriate leakage model would eliminate intra-function leakage.

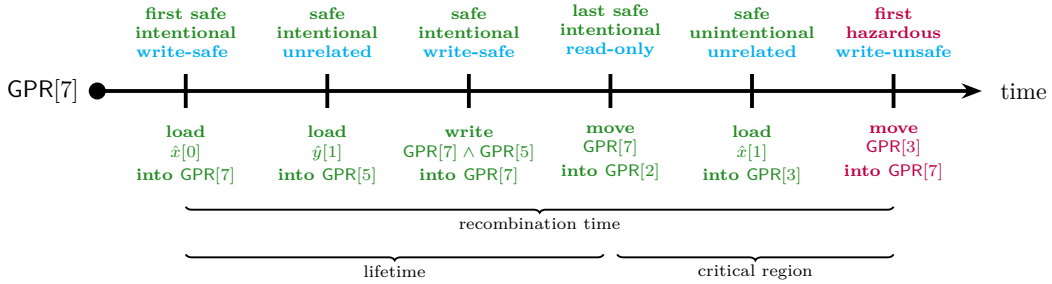
Definition 9 (Inter-function leakage). *Inter-function leakage is any leakage caused by the recombination of residual shares from function $\hat{f}_i$, during the computation of function $\hat{f}_j$, with $i \neq j$.*



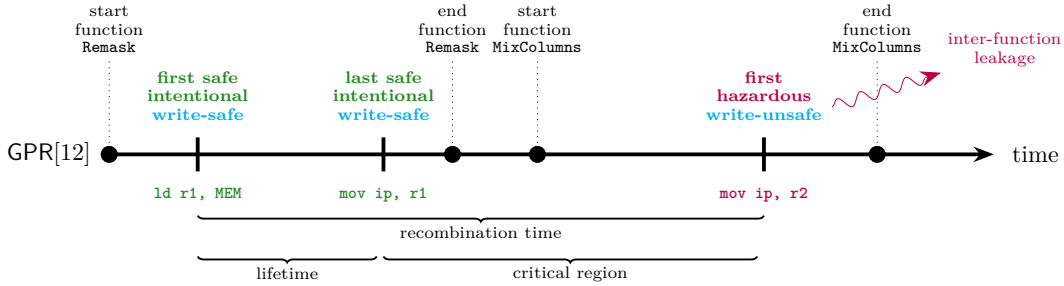
(a) Leakage timeline of a single share stored in GPR[i]. The span from first instruction to first hazardous instruction is the recombination time. The span from first to last intentional instruction is the lifetime, after which the share becomes residual. GPR[i] is erased during the critical region.



(b) Leakage timeline of two shares $\hat{x}[0]$, $\hat{x}[1]$ stored in GPR[i]. From left to right, the earliest hazard encountered causes intra-function leakage of $\hat{x}[0]$. The following hazard encountered causes inter-function leakage, since the share $\hat{x}[1]$ spills over from function $\hat{f}_0$ to function $\hat{f}_1$.



(c) Leakage timeline of share $\hat{x}[0]$ stored in GPR[7]. The share $\hat{x}[0]$ becomes residual after the first read-only instruction. The hazardous move instruction causes an overwrite that leaks $(\hat{x}[0] \wedge \hat{y}[1]) \oplus \hat{x}[1]$.



(d) Leakage timeline of share m_1, fetched initially from memory to `r1` and stored finally in register GPR[12] ≡ `ip`. The share m_1 becomes residual at the end of the `Remask` function, yet persists into the following `MixColumns` function, where a hazardous inter-function overwrite leaks $(x \oplus m_1) \oplus (y \oplus m_2) \oplus m_1$, combining shares m_1, m_2 (based on use of notation matching [MOP07]).

Figure 1: Diagrammatic explanations of concepts in Section 3.

Assuming provably secure and theoretically composable $\hat{f}_i$, $\hat{f}_j$, inter-function leakage is typically caused by practical composability problems that arise during the interaction between different functions (violation of Assumption 2, Section 1). Using practically composable functions that prevent the spillover of residual shares and anticipate the interactions between calleer and callee functions would eliminate inter-function leakage.

3.3 Overwrite effect

Every residual share recombination effect (during intra- or inter-function leakages) is triggered under different conditions, i.e., the definition of *safe* and *hazardous* instructions changes based on the effect. When considering overwrite-based share recombination with a share stored in register GPR[i], we can distinguish between safe and hazardous instructions based on their read/write behavior using the following conditions. We also exemplify these conditions in Figure 1c, which shows operations on share families $\hat{x}, \hat{y}$ that exhibit overwrite leakage on register GPR[7], with residual share $\hat{x}[0]$.

- **Safe instructions** are encapsulated in three cases: 1) *unrelated* instructions, i.e., instructions that do not have GPR[i] as part of their encoding, thus no overwrite is possible, 2) *read-only* instructions that have GPR[i] encoded as one of their source operands and not as a destination operand, thus unable to alter the contents of GPR[i], and 3) *write-safe* instructions that have GPR[i] encoded as a destination operand (and optionally also as a source operand), with such instructions altering the contents of GPR[i] without overwriting shares of the same family.

- **Hazardous instructions** are contained in a single case: *write-unsafe* instructions that have GPR[i] encoded as a destination operand (and optionally also as a source operand), with such instructions altering the contents of GPR[i] and overwriting the share in the register with another of the same family.

3.4 Rosita example

To better frame the concepts of residual shares and the associated inter-function leakage from overwrite effects, we analyse an example using the Rosita tool of Shelton et al. [SSB+21, Section V.C]. We demonstrate the share lifetime framework using the AES-128 test case[4], which implements the table-based masking scheme originally presented in [MOP07, Pages 228–231] and [HOM06]: within this Section, we use notation matching [MOP07] for consistency. [MOP07, Figure 9.1] offers a high-level overview of the masking scheme, whose application to a round of AES involves 6 masks: m and m' are the input and output masks for `SubBytes` function respectively, while m_1, m_2, m_3, and m_4 are the input masks for `MixColumns` function.

The Rosita example showcases a practical composability failure (violation of Assumption 2, Section 1), caused by the compiler usage of the scratch register GPR[12] ≡ `ip`. AAPCS [AAP23, Page 18], the ARM function[5] call convention, states that `ip` "*may be used by a linker as a scratch register between a routine and any [function] it calls*" but "*can also be used within a routine to hold intermediate values between [function] calls*".

The practical composability issue arises during the execution of `aes128`, in particular, during the call to the `Remask` function which refreshes the cipher state and is visible in Figure 1d. At the end of `Remask`, the instruction `mov ip, r1`, i.e., GPR[12] ← GPR[1], is executed. The register GPR[1] contains mask m_1, thus on exit from `Remask` GPR[12] contains mask m_1 as well. The `Remask` function is followed by a call to the masked `MixColumns` function

[4] See https://github.com/0xADE1A1DE/Rosita/tree/master/TESTS/aes.

[5] Note that although ARM-based documentation mixes use of the terms "procedure" or "routine", we retain "function" for consistency.

and the execution of instruction `mov ip, r2`, i.e., GPR[12] ← GPR[2]. In the beginning of `MixColumns`, GPR[2] contains $(x \oplus m_1) \oplus (y \oplus m_2)$ and GPR[12] (still) contains mask m_1, resulting in leakage caused by the overwrite effect.

Fundamentally, this leakage is caused by the share m_1 in GPR[12] being retained across masked functions (namely from `Remask` into `MixColumns`), i.e., it corresponds to inter-function leakage. Retaining the mask m_1 in GPR[12] is both permitted by AAPCS and has a positive impact on efficiency. However, the register now contains a residual share and this has a negative impact on security. That said, the `mov ip, r2` instruction in `MixColumns` is interesting, in the sense that it occurs as part of a *pair* which can be (trivially) optimised by the mapping `mov ip, r2 ; mov r5, ip` ↦ `mov r5, r2`. It is not clear *why* the compiler and/or assembler does not apply an optimisation of this type, but doing so would avoid use of `ip` and therefore eliminate the associated leakage. Rosita can and does resolve the associated leakage by instead rewriting `mov ip, r2` ↦ `mov ip, r7 ; mov ip, r2`, i.e., randomising (instead of erasing) the content of `ip` before overwriting it.

4 Design

Adopting Definition 3 as an exemplar for how leakage stemming residual shares is mitigated in, or during development of a software implementation, we make two claims. First, identifying residual shares (Definition 3, Step 2) can be difficult; this difficulty is amplified by inadequacy of the ABI, representing the interface between functions. That is, rather than the ABI guaranteeing composability and tasking the developer with *intra*-function analysis only, more complex, more challenging *inter*-function (or whole-program) analysis is potentially required. Second, share erasure (Definition 3, Step 3) essentially means inserting an instruction GPR[$\mathcal{R}_i$] ← 0 for $0 \leq i < |\mathcal{R}|$, where $\mathcal{R}$ is a register set whose elements (i.e., registers) each store a residual share. Doing so may not be robust (e.g., due to optimisations such as dead code elimination, instruction scheduling, etc.), or efficient (e.g., if $|\mathcal{R}|$ is large): Papagiannopoulos and Veshchikov [PV17, Table 1] quote a fairly significant overhead for their implementation, for example.

Based on RISC-V as introduced in Section 2.5, this Section proposes two simple components which can help support solutions to such challenges.

4.1 Component 1: a share erasure policy

Using the right-most column of Table 1, both the input to (or arguments of) and output (or return values) from an ABI-compliant function are well-defined, in the sense that one can reason precisely about their location (e.g., in a register or on the stack), and responsibility for preservation and restoration across the function call. As an example, the RISC-V ABI [RV:23] states that "*arguments passed by reference may be modified by the callee*", thus delineating expected behaviour across function calls.

To support the same type of reasoning about residual shares (Definition 3, Step 2), we define an extended ABI which provides clear semantics regarding responsibility of either the caller and/or callee function to erase shares. The extension is a simply stated requirement (or guarantee, depending on your perspective) that "*when a function returns, there should be no residual shares present within the register file*". Any function performing a masked operation is therefore deemed responsible, before it returns, for erasing any residual shares produced by it which do not also represent an output from it. Put another way, the extension acts to embody the share lifetime framework from Section 3 within the ABI: the scenario captured in Figure 1d would not occur if the functions involved complied with the extended ABI, for example.

Note that even in situations where the relevant ABI cannot be formally extended, one can still view the associated requirement as an informal guideline or best practice

followed during development. Also note that one could consider an alternative approach that explicitly assigns caller- and callee-*erase* properties to registers as analogies to caller- and callee-*save* properties; doing so implies a more complex, less ABI-agnostic relationship between caller and callee, but potentially allows for a higher degree of optimisation.

4.2 Component 2: a share erasure mechanism

The challenge with respect to share erasure (Definition 3, Step 3) is fundamentally related to efficiency. To address this challenge we define an extended ISA, i.e., an ISE, which allows more efficient erasure of shares than via the ISA alone: one could view the ISE as minimising the overhead of compliance with the extended ABI.

4.2.1 Instruction class 1: implicit, inter-function share erasure

Goal. The goal of this instruction class is to support efficient *inter*-function share erasure, i.e., erasure of a set of residual shares during return from a callee function. One could view the intention as ensuring baseline compliance with the extended ABI in an implicit manner: the instruction class is special-purpose in the sense that the set of erased registers is fixed, thus demanding little to no explicit action by the developer.

Design. First, define a register set $\mathcal{R} = \langle 5, 6, 7, 12, 13, 14, 15, 16, 17, 28, 29, 30, 31 \rangle$ of all caller-save registers in the ABI. Second, recalling that `ret` is a pseudo-instruction, i.e., `ret` $\mapsto$ `jalr x0, x1, 0`, define an instruction

$$\texttt{sec.ret} \;\mapsto\; \textbf{for } i = 0 \textbf{ to } |\mathcal{R}| - 1 \textbf{ do } \mathsf{GPR}[\mathcal{R}_i] \leftarrow 0 \;;\; \texttt{ret}.$$

which overloads `ret` by (pre-)erasing each register in $\mathcal{R}$. Assuming ABI-compliant caller and callee functions, this is possible because the caller must be pessimistic about the content in any caller-save register: when the callee returns, the caller must assume said content was overwritten. Before execution of the caller resumes, i.e., as part of the return, it is therefore possible to erase content in caller-save registers with no functional impact.

Note that $\mathcal{R}$ does not include *all* caller-save registers; we deliberately do *not* include GPR[1] (return address)[6], GPR[10] (return value), and GPR[11] (return value), for example.

4.2.2 Instruction class 2: explicit, intra-function share erasure

Goal. The goal of this instruction class is to support efficient *intra*-function share erasure, i.e., erasure of a set of residual shares during execution of a callee function (and so interleaved with instructions from said function). One could view the intention as allowing complete compliance with the extended ABI (i.e., beyond the baseline offered by instruction class 1) in an explicit manner: the instruction class is general-purpose in the sense that the set of erased registers is configurable, but demands explicit action by the developer.

Design. Imagine that at some point within the callee function, we need to erase some set of registers denoted $\mathcal{R}$; the overhead of doing so naively will be $|\mathcal{R}|$ instructions using the ISA, implying a useful ISE should bound the overhead to $m < |\mathcal{R}|$ instructions. Conceptually at least, it seems that achieving the ideal of $m = 1$ is possible: let $\mathcal{R} = \langle 0, 1, \ldots, 31 \rangle$ capture the set of all registers, then define

$$\texttt{sec.zrr imm} \;\mapsto\; \textbf{for } i = 0 \textbf{ to } |\mathcal{R}| - 1 \textbf{ do if } \texttt{imm}_i = 1 \textbf{ then } \mathsf{GPR}[\mathcal{R}_i] \leftarrow 0,$$

[6] Note that `ret`, namely `jalr x0, x1, 0`, writes the value of GPR[1] into the program counter. This means when `sec.ret` is executed, GPR[1] must contain a valid return address and thus cannot hold a residual share.

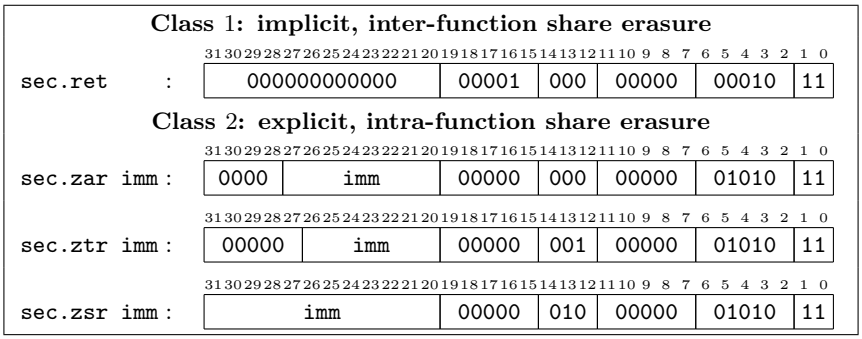


Figure 2: An example encoding, using the RISC-V I-type format, for instructions outlined in Section 4.2.1 and Section 4.2.2.

i.e., an instruction which can selectively erase any subset of registers by using an appropriate value for the immediate operand `imm`. Notice, however, that the encoding for `sec.zrr` requires an $|\mathcal{R}|$-bit immediate operand: this is not practical for RV32I given that $|\mathcal{R}| = 32$, and indeed most other base ISAs.

The natural way to address this impracticality is to partition the single, large register set into multiple, smaller register sets. From a positive perspective this compromise means smaller, more practical immediate operand sizes, but from a negative perspective it means a larger bound of some $1 < m < |R|$. A large design space of viable approaches exists, instances within which can be differentiated, e.g., by their trade-off features such as the register sets, number of instructions, and best-, worst-, and average-case overhead.

We opt for an approach which is specialised to suit the base ISA. Let $\mathcal{A} = \langle 10, 11, 12, 13, 14, 15, 16, 17 \rangle$, $\mathcal{T} = \langle 5, 6, 7, 28, 29, 30, 31 \rangle$, and $\mathcal{S} = \langle 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27 \rangle$, such that $|\mathcal{A}| = 8$, $|\mathcal{T}| = 7$, and $|\mathcal{S}| = 12$, capture the argument, temporary, saved register sets respectively (per Table 1). Then, define

$$\begin{aligned}
\texttt{sec.zar imm} &\mapsto \textbf{for } i = 0 \textbf{ to } |\mathcal{A}| - 1 \textbf{ do if } \texttt{imm}_i = 1 \textbf{ then } \mathsf{GPR}[\mathcal{A}_i] \leftarrow 0 \\
\texttt{sec.ztr imm} &\mapsto \textbf{for } i = 0 \textbf{ to } |\mathcal{T}| - 1 \textbf{ do if } \texttt{imm}_i = 1 \textbf{ then } \mathsf{GPR}[\mathcal{T}_i] \leftarrow 0 \\
\texttt{sec.zsr imm} &\mapsto \textbf{for } i = 0 \textbf{ to } |\mathcal{S}| - 1 \textbf{ do if } \texttt{imm}_i = 1 \textbf{ then } \mathsf{GPR}[\mathcal{S}_i] \leftarrow 0
\end{aligned}$$

This approach is motivated by the observation that a callee function is more likely to use temporary registers (i.e., those in $\mathcal{T}$) before saved registers (i.e., those in $\mathcal{S}$); in the best case the overhead is therefore $m = 1$ instruction, whereas in the worst case it becomes $m = 3$ instructions. Additionally, the immediate operand sizes involved are well-aligned with the RISC-V I-type format which can cater for 7-, 8-, or 12-bit values of `imm`; a 16-bit `imm` is, for example, more problematic[7].

Note that the union of $\mathcal{A}$, $\mathcal{T}$, and $\mathcal{S}$ does not cover *all* registers: we deliberately do *not* include $\mathsf{GPR}[0]$ (zero), $\mathsf{GPR}[1]$ (return address), $\mathsf{GPR}[2]$ (stack pointer), $\mathsf{GPR}[3]$ (global pointer), nor $\mathsf{GPR}[4]$ (thread pointer), for example.

4.2.3 Instruction encoding

RV32I employs a fixed-length, 32-bit instruction encoding with four base instruction formats [RV:19, Figure 2.2] (plus variants for, e.g., immediate operands). Per Figure 2, we use the I-type format to encode instructions in the ISE.

[7] Although the U-type format allows a 20-bit `imm`, this option is unattractive because it consumes a major opcode (i.e., a large part of the overall instruction encoding space) per instruction. Another option could be to use a variable-length instruction format [RV:19, Section 1.2], but we rule this out because it leads to a strong requirement (i.e., is a special, somewhat niche case) with respect to the micro-architectural implementation.

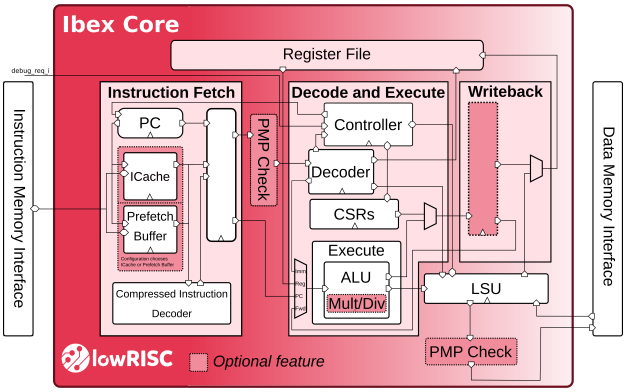


Figure 3: A block diagram describing the Ibex micro-architecture (image source: `blockdiagram.svg`, obtained from `https://github.com/lowRISC/ibex`).

5 Implementation

This Section, describes a prototype implementation of the design concept outlined in Section 4 from two perspectives. First (in Section 5.1), the hardware-perspective focuses on alteration of a base core to yield an ISE-enabled extended core with share erasure capabilities. Then, second (in Section 5.2), the software-perspective focuses on development of hardened RV32I-based gadget implementations.

5.1 Hardware

Base core. Ibex[8], a popular 32-bit RISC-V core designed for embedded environments, is used as the base core in this work. It supports either the integer (i.e., RV32I) or embedded (i.e., RV32E) RISC-V base ISA, and can be supplemented by the multiplication (M) [RV:19, Chapter 7], compressed (C) [RV:19, Chapter 16], or bit manipulation (B) [RV:19, Chapter 17] extensions. In detail, we develop the prototype ISE implementation and perform our experiments using the Ibex Demo System[9], which comprises the Ibex core plus peripherals such as a UART. The flip-flop-based register file implementation is selected, and the default configuration is used for other settings: the ISA is RV32IMC and a 2-stage pipeline is used (including a fetch stage, plus a decode and execute stage).

Extended core. To implement the ISE, we adapt the decoder to 1) identify the 4 additional instructions, and 2) generate a signal `rf_erase` which indicates the set of registers to be erased. Subsequently, `rf_erase` is connected directly to the register file, with each bit controlling the reset input of a corresponding flip-flop:

- In the implementation of `sec.ret`, the value for `rf_erase` is hard-coded (because the set of registers to be erased is fixed). Then, recall that `ret` is a pseudo-instruction mapping to `jalr x0, x1, 0`. We therefore select an I-type encoding for `sec.ret` such that `rd` = `x0`, `rs1` = `x1`, and `imm` = 0: doing so allows `sec.ret` to reuse logic related to `jalr` in order to perform a function return.
- In the implementation of `sec.zar`, `sec.ztr`, and `sec.zsr`, the value for `rf_erase` is determined by the immediate `imm`.

[8] `https://github.com/lowRISC/ibex`
[9] `https://github.com/lowRISC/ibex-demo-system`

```
.macro sec.ret
  .insn i CUSTOM_0, 0, x0, x1, 0
.endm
.macro sec.ztr imm
  .insn i CUSTOM_1, 1, x0, x0, \imm
.endm

#if   defined( SCRUB_1_NONE )
.macro SCRUB_1
.endm
#elif defined( SCRUB_1_ISA  )
.macro SCRUB_1
  xor t0, x0, x0 ; xor t2, x0, x0
.endm
#elif defined( SCRUB_1_ISE  )
.macro SCRUB_1
  sec.ztr 0x05
.endm
#endif

#if   defined( SCRUB_2_NONE )
.macro SCRUB_2
  ret
.endm
#elif defined( SCRUB_2_ISA  )
.macro SCRUB_2
  xor t0, x0, x0 ; xor t1, x0, x0 ; xor t2, x0, x0 ; xor t3, x0, x0
  ret
.endm
#elif defined( SCRUB_2_ISE  )
.macro SCRUB_2
  sec.ret
.endm
#endif
```

Figure 4: A set of macros which allow the function in Figure 5 to 1) use of instructions defined by the ISE, and 2) apply none, ISA-based, and ISE-based erasure strategies.

```
gadget: lw  t0, 0x00(a2) // Alg. 5, Line  1: load(R4,R2,0)
        lw  t1, 0x00(a1) // Alg. 5, Line  2: load(R5,R1,0)
        and t0, t0, t1   // Alg. 5, Line  3: and(R4,R5)
        lw  t2, 0x00(a3) // Alg. 5, Line  4: load(R6,R3,0)
        xor t2, t2, t0   // Alg. 5, Line  5: xor(R6,R4)
        lw  t3, 0x04(a2) // Alg. 5, Line  6: load(R7,R2,1)
        and t1, t1, t3   // Alg. 5, Line  7: and(R5,R7)
        xor t1, t1, t2   // Alg. 5, Line  8: xor(R5,R6)
        sw  t1, 0x00(a0) // Alg. 5, Line  9: store(R5,R0,0)
        SCRUB_1          // Alg. 5, Line 10: scrub(R4,R6)
        lw  t0, 0x04(a1) // Alg. 5, Line 11: load(R4,R1,1)
        and t3, t3, t0   // Alg. 5, Line 12: and(R7,R4)
        lw  t2, 0x00(a3) // Alg. 5, Line 13: load(R6,R3,0)
        xor t2, t2, t3   // Alg. 5, Line 14: xor(R6,R7)
        lw  t1, 0x00(a2) // Alg. 5, Line 15: load(R5,R2,0)
        and t1, t1, t0   // Alg. 5, Line 16: and(R5,R4)
        xor t2, t2, t1   // Alg. 5, Line 17: xor(R6,R5)
        sw  t2, 0x04(a0) // Alg. 5, Line 18: store(R6,R0,1)
        SCRUB_2          // Alg. 5, Line 19: scrub(R4,R5,R6,R7) + return
```

Figure 5: A translation of the 1-st order, provably secure multiplication gadget of Barthe et al. [BGG+21, Algorithm 5] to RV32I: the gadget implements an $\hat{f}_i$ associated with $r = f_i(x, y) = x \wedge y$, using an $\hat{r} = \langle \hat{r}[0], \hat{r}[1] \rangle$, $\hat{x} = \langle \hat{x}[0], \hat{x}[1] \rangle$, and $\hat{y} = \langle \hat{y}[0], \hat{y}[1] \rangle$, whose base addresses are in $\mathsf{GPR}[10] \equiv \mathtt{a0}$, $\mathsf{GPR}[11] \equiv \mathtt{a1}$, and $\mathsf{GPR}[12] \equiv \mathtt{a2}$ respectively.

5.2 Software

Our starting point is the extension of scVerif to support RV32I. Our approach for doing so is generic in the sense that it is not informed by a specific micro-architecture, and minimalist in the sense that we only model features pertinent to our scope. For example, given our scope we 1) include the minimal set of instructions required (namely `lw`, `sw`, `and`, and `xor`) in the instruction model, using standard (value-based) and transient (overwrite-based) leakage models, 2) exclude leakage effects stemming from memory access from the leakage model, and 3) exclude other leakage effects stemming from the micro-architecture (e.g., operand or pipeline leakage) from the leakage model, and therefore also ignore instances of the Clear macro because these relate to micro-architectural resources. Next, we translate several provably secure gadgets used by Barthe et al. [BGG+21] from ARMv6-M into RV32I implementations (see [BGG+21, Algorithms 4,5,6,8,17,18,19,20,21,22]).

We opt for a direct translation rather than instrumenting RV32I-specific optimisations, applying 5 major[10] steps. First, we map ARMv6-M instructions into their RV32I equivalent: this is possible due to the instructions used and broad similarity between the ISAs. Second, because leakage stemming from micro-architectural resources is out of scope, we remove (or ignore) all Clear macros. Third, since we implement each gadget as a function that complies with the extended ABI, we append a return instruction. Fourth, we translate[11] the register allocation, i.e., map `R0` through `R3` to `a0` through `a3`, and map `R4` through `R7` to `t0` through `t3`. Fifth, we instantiate all Scrub macros based on one of three erasure strategies: these are no erasure (or none), ISA-based erasure, and ISE-based erasure. For the no erasure case, Scrub macros are simply removed; for ISA-based erasure they are replaced with a `xor` instruction which writes zero into the register. For ISE-based erasure, they are replaced by applying the following process: first, the return instruction plus any preceding sequence of Scrub macros is collapsed into a `sec.ret` instruction, then any remaining sequence of Scrub macros is collapsed into a `sec.ztr` instruction. The resulting software implementation is compiled using version 10.2.0 of the lowRISC-packaged, GCC-based RISC-V tool-chain[12], using the `.insn` directive to support use of ISE-based instructions where applicable.

Consider [BGG+21, Algorithm 5] as an example, the RV32I translation of which is demonstrated in Figure 4 (the supporting macros) and Figure 5 (the implementation). Lines 1 to 6 of Figure 4 define macros that allow use of ISE-based instructions. Lines 8 to 34 of Figure 4 define macros that independently capture the three possible erasure strategies as invoked in two places within the algorithm. `SCRUB_1` implements line 10 of [BGG+21, Algorithm 5], i.e., Scrub(`R4`), Scrub(`R6`). `SCRUB_2` implements line 19, i.e., Scrub(`R4`), Scrub(`R5`), Scrub(`R6`), Scrub(`R7`), *plus* a return instruction. Lines 1 to 19 of Figure 5 define the gadget function itself: for clarity, we present this in terms of a left-hand side, i.e., the RISC-V implementation itself, and a right-hand side, i.e., comments capturing correspondence with the original algorithm.

6 Evaluation

This Section uses the prototype implementation outlined in Section 5 as a vehicle to evaluate the general design concept of Section 4. We do so using two main metrics: these are 1) overhead, i.e., efficiency as decomposed into hardware- (namely area; see Section 6.2)

[10] For brevity, we omit minor alterations required to align each listing in [BGG+21] with the associated, concrete implementation in https://github.com/scverif/gadgets.

[11] Note that Barthe et al. [BGG+21, Section 4.1] adopt a relaxed approach to register allocation, in the sense that "*registers R4, R5, R6, and R7 are used to perform the elementary operations*" whereas "*registers beyond R7 are used rarely*". We retain this approach, meaning the (positive or negative) impact of using a wider set of registers is not explored even though doing so is possible.

[12] https://github.com/lowRISC/lowrisc-toolchains/releases/tag/20220524-1

Table 2: A comparison of area, i.e., hardware overhead, covering the base core and the extended core, i.e., base core + ISE.

	Registers	LUTs
Base core	2361 (1.00×)	3718 (1.00×)
Base core + ISE	2361 (1.00×)	4360 (1.17×)

Table 3: A comparison of execution latency (measured in cycles) for various gadgets and erasure strategies applied within them; each gadget is as presented by Barthe et al. [BGG+21], then translated to RV32I using the process outlined in Section 5.2.

	No erasure	ISA-based erasure	ISE-based erasure
[BGG+21, Algorithm 4] ($d = 1$, addition)	10	13 (1.30×)	10 (1.00×)
[BGG+21, Algorithm 5] ($d = 1$, multiplication)	19	25 (1.32×)	20 (1.05×)
[BGG+21, Algorithm 6] ($d = 2$, addition)	14	18 (1.29×)	15 (1.07×)
[BGG+21, Algorithm 8] ($d = 2$, multiplication)	47	63 (1.34×)	55 (1.17×)
[BGG+21, Algorithm 17] ($d = 1$, CALCA_OPT)	23	26 (1.13×)	23 (1.00×)
[BGG+21, Algorithm 18] ($d = 1$, CALCB_OPT)	29	32 (1.10×)	29 (1.00×)
[BGG+21, Algorithm 19] ($d = 1$, CALCG_OPT)	65	70 (1.08×)	66 (1.02×)
[BGG+21, Algorithm 20] ($d = 2$, CALCA_OPT)	34	37 (1.09×)	34 (1.00×)
[BGG+21, Algorithm 21] ($d = 2$, CALCB_OPT)	68	82 (1.21×)	74 (1.09×)
[BGG+21, Algorithm 22] ($d = 2$, CALCG_OPT)	143	161 (1.13×)	148 (1.03×)

and software-oriented (namely execution latency; see Section 6.3) components and 2) security (see Section 6.4), i.e., effectiveness with respect to leakage elimination.

6.1 Experimental platform

We combined a NewAE ChipWhisperer CW305[13] board and a NewAE ChipWhisperer CW1173 (or ChipWhisperer-Lite)[14] board to provide an experimental platform. We used Xilinx Vivado 2020.1 to synthesise either the base or extended core for the Xilinx Artix-7 (model `XC7A100T2FTG256`) FPGA device on the CW305; once programmed onto the FPGA, we supplied the core with an 8 MHz clock frequency as generated by the CW1173. We attached the CW1173 to the `X4` pin of the CW305. This implies the measured signal was amplified by Low-Noise Amplifiers (LNAs) on both CW305 (fixed to 20 dB gain) and CW1173 (set to 20 dB gain) boards; to minimise measurement noise, an external Power Supply Unit (PSU) was used to supply the FPGA with a 1 V power supply.

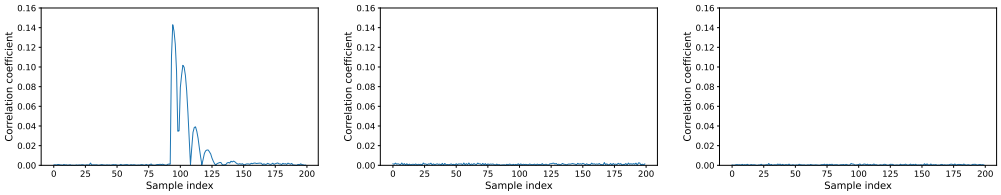
6.2 Evaluating area

Table 2 reports the area of base and extended cores stemming from their synthesis. Relative to the base core core, the extended core requires 17% more LUTs and 0% more registers, i.e., the same number, which is expected because there is no additional state.

6.3 Evaluating latency

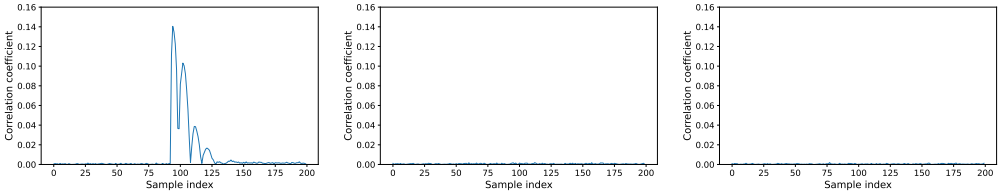
On a per-instruction basis, `sec.ret` has a 2-cycle execution latency (matching `ret`) whereas `sec.zar`, `sec.ztr`, and `sec.zsr` have a 1-cycle execution latency. On a per-gadget basis, Table 3 reports the execution latency of RV32I translations of various gadgets presented by Barthe et al. [BGG+21] for three erasure strategies. This broader perspective is important because the advantage of an ISE-based over an an ISA-based erasure strategy is heavily

[13] https://rtfm.newae.com/Targets/CW305ArtixFPGA
[14] https://rtfm.newae.com/Capture/ChipWhisperer-Lite



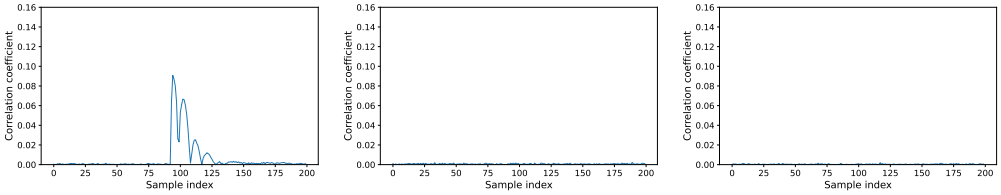
(a) No erasure. (b) ISA-based erasure. (c) ISE-based erasure.

Figure 6: Case 1: intra-gadget, $\mathcal{M} = \mathrm{HD}(\hat{x}[0] \wedge \hat{y}[0], \hat{x}[1])$, constrained such that $\hat{y}[0] = \mathbf{1}^{32}$.



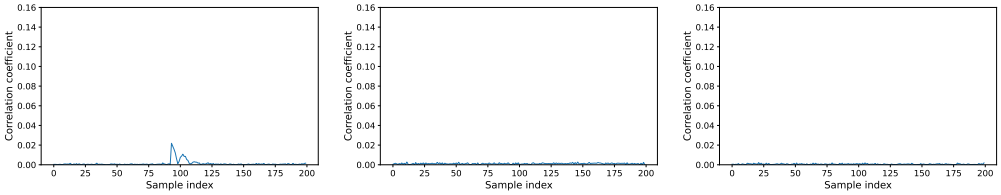
(a) No erasure. (b) ISA-based erasure. (c) ISE-based erasure.

Figure 7: Case 2: inter-gadget, $\mathcal{M} = \mathrm{HD}(\hat{x}[1], \hat{x}[0])$.



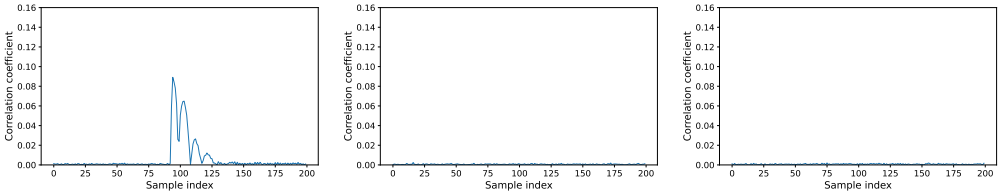
(a) No erasure. (b) ISA-based erasure. (c) ISE-based erasure.

Figure 8: Case 2: inter-gadget, $\mathcal{M} = \mathrm{HD}(\hat{x}[1] \wedge \hat{y}[0], \hat{x}[0])$, constrained such that $\hat{y}[0] = \mathbf{1}^{32}$.



(a) No erasure. (b) ISA-based erasure. (c) ISE-based erasure.

Figure 9: Case 2: inter-gadget, $\mathcal{M} = \mathrm{HD}(\hat{r}[1], \hat{r}[0])$.



(a) No erasure. (b) ISA-based erasure. (c) ISE-based erasure.

Figure 10: Case 2: inter-gadget, $\mathcal{M} = \mathrm{HD}(\hat{x}[1] \wedge \hat{y}[1], \hat{x}[0])$, constrained such that $\hat{y}[1] = \mathbf{1}^{32}$.

dependent on factors such as 1) how often the SCRUB macro is invoked, 2) whether or not multiple such invocations are sequential, and 3) which registers each invocation involves.

The top part of Table 3 deals with smaller addition and multiplication gadgets. ISA-based erasure increases execution latency between a factor of 1.29× and 1.34×, with ISE-based erasure reducing this significantly (in the best case, implying no overhead at all). The bottom part of Table 3 deals with larger gadgets that support masked implementation of the PRESENT S-box: [BGG+21, Algorithm 13] invokes CALCB_OPT once, CALCG_OPT twice, and CALCA_OPT once, to compute the masked S-box output from an input. ISA-based erasure increases execution latency between a factor of 1.08× and 1.21×, with ISE-based erasure again reducing this significantly (in the best case, implying no overhead at all).

6.4 Evaluating security

To evaluate the security impact of share erasure, we focus on the gadget shown in Figure 5, i.e., the RV32I-based translation of [BGG+21, Algorithm 5] per Section 5.2. To do so we employ the combination of 1) a formal verification approach based on use of (our extensions to) SCVERIF followed by 2) an experimental validation, but stress the focus is assessment rather than exploitation of leakage.

Approach. Motivated by 1) evidence such as [SACB24] with respect to the leakage characteristics of the base core, and 2) our specific focus on transitional leakage stemming from the overwrite effect in the register file, we opted *not* to use Test Vector Leakage Assessment (TVLA) [GJJR11] methodology: per Standaert [Sta19], we viewed the potential for false negatives and positives as problematic. Instead, we use Correlation Power Analysis (CPA) [BCO04]: we correlate power traces with the leakage model $\mathcal{M} = \mathrm{HD}(\beta, \alpha)$ for the value β before (resp. α after) update of a given register, thereby using a Hamming Distance (HD) leakage model to reflect transitional leakage.

We consider two experiments which both stem from problematic instruction sequences identified by SCVERIF: experiment 1 deals with *intra*-gadget leakage, whereas experiment 2 deals with *inter*-gadget leakage. Although relevant to *both* experiments, consider experiment 1 for example. In line 11 of Figure 5, if no erasure is performed then $\mathsf{GPR}[5] \equiv \texttt{t0}$ holding $\beta = \hat{x}[0] \wedge \hat{y}[0]$ is overwritten with $\alpha = \hat{x}[1]$; per SCVERIF and our definition of hazardous instructions, this is problematic. In practice, however, the situation is more nuanced. Only if $\hat{y}[0] = \mathbf{1}^{32}$ will this overwrite leak the full recombination of $\hat{x}[0]$ and $\hat{x}[1]$; otherwise less information is leaked, with the extreme case of $\hat{y}[0] = \mathbf{0}^{32}$ leaking no information at all.

We therefore construct a micro-benchmark, focused on some target instruction which overwrites β with α. We execute the micro-benchmark with constrained random $\hat{x}$ and $\hat{y}$ as input, acquiring 2 million traces then used to compute the associated correlation coefficient. The constraint maximises potential leakage per the argument above, therefore producing a worst case (resp. best case) as viewed from the perspective of the countermeasure (resp. attacker): our general hypothesis is that transitional (overwrite-based) leakage will be observable (i.e., a peak will occur in the correlation coefficient) without share erasure, but will *not* be observable (i.e., a peak will not occur in the correlation coefficient) with share erasure realised, using either an ISA- or ISE-based approach. Note that all resulting plots (e.g., Figure 6) are centered on the target instruction, meaning any peak occurs at the same sample index.

Experiment 1: intra-gadget. In this case we evaluate *intra*-gadget leakage, i.e., that which stems from overwriting "inside" $n = 1$ gadget. In concrete terms, the gadget is described by Figure 5. The target instruction is `lw t0, 0x04(a1)` at line 11: it loads $\alpha = \hat{x}[1]$ into and so overwrites $\mathsf{GPR}[5]$, which holds $\beta = \hat{x}[0] \wedge \hat{y}[0]$. If `SCRUB_1_NONE`

is defined then no erasure strategy is realised and we expect leakage (because the load will overwrite β with α causing share recombination); if `SCRUB_1_ISA` or `SCRUB_1_ISE` is defined then an erasure strategy is realised during the critical region and we expect no leakage (because the load will overwrite 0 with α).

This hypothesis is validated theoretically by use of scVerif, which identifies the target instruction as problematic, then experimentally per Figure 6: the plot shows that the `sec.ztr` instruction efficiently and effectively prevents said leakage.

Experiment 2: inter-gadget. In this case we evaluate *inter*-gadget leakage, i.e., that which stems from overwriting "between" $n > 1$ gadgets. To do so, we consider $n = 2$ and so execution of a callee gadget as invoked by a caller gadget. The caller gadget is (artificially) constructed to intentionally overwrite all (potentially) residual shares stemming from the callee gadget: the idea is that the caller gadget acts as worst-case "canary", in the sense that it will intentionally expose any practical composability failure. In concrete terms, we use Figure 5 as the callee gadget. The target instruction is `lw t0, 0x00(a1)` within the caller gadget (and so executed after the callee returns): it loads $\alpha = \hat{x}[0]$ into and so overwrites GPR[5], which holds $\beta = \hat{x}[1]$. Having defined `SCRUB_1_ISA` to exclude intra-gadget leakage, if `SCRUB_2_NONE` is defined then no erasure strategy is realised and we expect leakage (because the load will overwrite β with α causing share recombination); if `SCRUB_2_ISA` or `SCRUB_2_ISE` is defined then an erasure strategy is realised during the critical region and we expect no leakage (because the load will overwrite 0 with α).

This hypothesis is validated theoretically by use of scVerif, which identifies the target instruction as problematic, then experimentally per Figure 7 (noting similar cases in Figure 8, Figure 9, and Figure 10 for other residual shares): specifically, the plot demonstrates use of the `sec.ret` instruction efficiently and effectively prevents said leakage.

7 Discussion

Generalisations: other architectures (i.e., ISAs). Although one can imagine realising the underlying design concepts from Section 4 as an extension to any base ISA, various features of said base ISA will have an impact. To illustrate this claim, we explore the alternative case of ARMv6-M as an example motivated by Section 3.4. Some specific examples are as follows. First, it exposes the program counter as `pc` $\equiv$ GPR[15]. This allows *any* instruction to update the program counter; although a function will likely return using an idiom such as `bx lr` or `mov pc, lr` if the link register `lr` $\equiv$ GPR[14] stores the return address, there is no dedicated return (pseudo-)instruction. Second, the calling convention outlined in AAPCS [AAP23, Page 18] has more special-cases (the `ip` register outlined in Section 3.4 is one example), but, equally, defines fewer caller-save register: this places more emphasis on the callee to preserve register content, and therefore less potential for such a callee to return with residual shares resident in the register file. Third, the smaller, 16-element register file is less demanding of the immediate operand used to encode a register set: given the instruction formats available, it is plausible to use an analogue of the `sec.zrr` candidate described in Section 4.2.2, for example.

Generalisations: other micro-architectures. Although one can imagine realising the underlying design concepts from Section 4 in any base core, i.e., micro-architecture, various features of said base core will have an impact. For example, we used Ibex as a base core, which, in relative terms, has a simple micro-architecture: another, e.g., more complex micro-architecture will likely lead to different challenges with respect to both efficiency and security. Some specific examples are as follows. First, the ISE instructions potentially write into multiple target registers. Although this behaviour is special-purpose (they

Table 4: A classification of and qualitative comparison with some alternative approaches.

Approach	Invocation	Invocation	Remit
This work	manual	conditional	prevention
Cheng et al. [CPW24]	automatic	unconditional	prevention
Escouteloup and Migliore [EM25]	automatic	unconditional	prevention
Taint analysis: compile-time (in software)	manual	conditional	detection
Taint analysis: run-time (in hardware)	automatic	conditional	detection

write zero, so each register can be reset rather than require a dedicated write port), it complicates mechanisms such as forwarding; in turn, this fact may lead to increased area overhead. Second, out-of-order, e.g., super-scalar, execution presents a number of challenges. Although we stress said challenges impact both ISA- and ISE-based erasure strategies, examples include the fact that 1) one can longer guarantee any share erasure occurs in program order (which might be addressed using some form of fence; cf. Gao et al. [GMPP20]), and 2) consideration of the management of physical versus logical registers may be necessary (cf. Desmet, Kundu, and Verbauwhede [DKV26]).

Generalisations: other programming constructs. It is common to implement gadgets using macros rather than functions, since doing so eliminates function call overhead (which can be significant for short instruction sequences); catering specifically for this idiom therefore has some value. Focusing on assembly language macros supported by GAS[15] as an example, two specific differences are important. First, macros do not have a return per se. It is reasonable to assume gadgets are straight-line instruction sequences, however, and therefore the macro start (resp. end) represents the entry (resp. exit, or return) point. Second, although they do have arguments, macros do not have annotated input and output. As a result, the analogous requirement "*at the end of a macro, there should be no residual shares (i.e., shares produced within the macro but do not relate to an argument) present within the register file*" is weaker but similar to that for functions.

Alternative approaches. Even considering approaches with the same scope, a large design space of alternatives exist. At a high level, one can classify instances via (at least) three properties: these capture whether the instance 1) is automatically or manually invoked, 2) is conditionally (or "opt-in") versus unconditionally invoked (or "always-on"), and 3) focuses on detection (identifying the need for share erasure) versus prevention (implementing share erasure). Table 4 uses this classification to highlight selective instances, and compare them with our work. First, Cheng et al. [CPW24] present a mechanism which prevents overwrite-based leakage in the register file; it uses light-weight register renaming to ensure all writes are to a pre-zeroed register, which Escouteloup and Migliore [EM25] subsequently also consider. Note that [CPW24] *also* considers a mechanism for preventing overwrite-based leakage in memory; when invoked this requires some level of manual control. Second, the general concept of taint analysis relates to tracking the use and propagation of data (e.g., from some source, to some sink operation) to detect violations of some security policy; it can be realised at compile-time or at run-time. Both cases require a means of identifying pertinent data, i.e., shares; both cases can be extended to prevention-based by considering some associated action, i.e., share erasure, triggered by detection. Use at compile-time would represent a similar workflow and outcome as use of scVerif; at run-time, it would require hardware support for 1) management of additional meta-data to, e.g., tag registers containing shares, and 2) share erasure, i.e., zeroisation of registers when the analysis highlights an instance of insecure overwriting.

[15] See, e.g., https://sourceware.org/binutils/docs/as/Macro.html.

Other related work. Li et al. [LHP20] discuss support for flushing (micro-)architectural state: execution of the `flushx` instruction, can, for example, zeroise all registers in the register file. They focus on 1) zeroisation in support of temporal isolation, using 2) unconditional flushing of *all* registers; we focus on 1) zeroisation in support of robust masking, using 2) conditional erasure of *some* specified registers. Olmos et al. [OBG+24] address the challenge of zeroisation by leveraging the Jasmin [ABB+17] programming language and compiler tool-chain. They focus on 1) direct recovery of secrets, 2) data stored in memory, and 3) an *inter*-function call granularity; in contrast, we focus on 1) indirect recovery of secrets (via side-channel leakage), 2) data stored in the register file, and 3) both *inter*- and *intra*-function call granularity. Gigerl et al. [GPM23] focus on security implications that stem from executing a masked implementation on a managed (versus a "bare-metal") platform, i.e., when an operating system kernel exists. They focus on 1) the system call convention, and 2) elimination of inter-process leakage which might stem from an *inter*-process context switch; in contrast, we focus on 1) the function call convention, and 2) elimination of intra- and inter-function leakage which might stem from an *intra*-process function call.

8 Conclusion

Within the context of cryptographic software implementation, elimination of leakage represents a fundamentally important challenge. In this paper we address one component of said challenge, relating to practical model and composability failures within the context of masking and, specifically, otherwise theoretically secure gadgets. In short, our goal was to ensure that an implementation of some gadget $\hat{f}_i$ can be freely composed with, e.g., called by another $\hat{f}_j$ without causing leakage not catered for by the security proof for $\hat{f}$.

To address this challenge, we focused on foundational design concepts which can be used to support erasure of residual shares related to $\hat{f}_i$, thereby preventing intra- and inter-function leakage. Framed within the context of Barthe et al. [BGG+21], and used in combination, our two proposed components represent an effective and efficient way to implement instances of their Scrub macro, and a natural complement to alternatives (such as FENL [GMPP20]) that implement instances of their Clear macro. Although the design and implementation of both components is dependant on the base ISA and base core (i.e., micro-architecture) used, we stress that the underlying framework concepts are of a general nature. Finally, although we focus on the overwrite effect, the use of both components has potentially more general impact, e.g., on related leakage effects, such as neighbour, operand and pipeline effects that residual shares can also unintentionally induce.

Acknowledgements

This work has been supported in part by EPSRC via grant EP/R012288/1 under the RISE (http://www.ukrise.org) programme, and Innovate UK via project 10065634 (SCHEME: Safety Critical Harsh Environment Micro-processing Evolution), in part by the National Natural Science Foundation of China (Grant No. 62502275), in part by a UvA Starter Grant, and in part by the NWO CiCS (Challenges in Cyber Security) gravitation project.

References

[AAP23] Procedure call standard for the Arm architecture. Technical Report 2023Q1, ARM Ltd., 2023. https://github.com/ARM-software/abi-aa.

[ABB+17] J. Bacelar Almeida, M. Barbosa, G. Barthe, A. Blot, B. Grégoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub. Jasmin: High-assurance and high-speed cryptography. In *Computer and Communications Security (CCS)*, pages 1807–1823, 2017. https://doi.org/10.1145/3133956.3134078.

[BCM+25] S. Belaïd, G. Cassiers, C. Mutschler, M. Rivain, T. Roche, F.-X. Standaert, and A.R. Taleb. SoK: A methodology to achieve provable side-channel security in real-world implementations. *IACR Communications in Cryptology*, 2(1), 2025. https://doi.org/10.62056/aebngy4e-.

[BCO04] E. Brier, C. Clavier, and F. Olivier. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 3156, pages 16–29. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-28632-5_2.

[BDLU17] A. Biryukov, D. Dinu, Y. Le Corre, and A. Udovenko. Optimal first-order Boolean masking for embedded IoT devices. In *Smart Card Research and Advanced Applications (CARDIS)*, LNCS 10728, pages 22–41. Springer-Verlag, 2017. https://doi.org/10.1007/978-3-319-75208-2_2.

[BGG+21] G. Barthe, M. Gourjon, B. Grégoire, M. Orlt, C. Paglialonga, and L. Porth. Masking in fine-grained leakage models: Construction, implementation and verification. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2021(2):189–228, 2021. https://doi.org/10.46586/tches.v2021.i2.189-228.

[BWG+22] A. Beckers, L. Wouters, B. Gierlichs, B. Preneel, and I. Verbauwhede. Provable secure software masking in the real-world. In *Constructive Side-Channel Analysis and Secure Design (COSADE)*, LNCS 13211, pages 215–235. Springer-Verlag, 2022. https://doi.org/10.1007/978-3-030-99766-3_10.

[CPGR05] J. Chow, B. Pfaff, T. Garfinkel, and M. Rosenblum. Shredding your garbage: Reducing data lifetime through secure deallocation. In *USENIX Security Symposium*, pages 331–346, 2005. https://www.usenix.org/conference/14th-usenix-security-symposium/shredding-your-garbage-reducing-data-lifetime-through.

[CPW24] H. Cheng, D. Page, and W. Wang. eLIMInate: a leakage-focused ISE for masked implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(2):329–358, 2024. https://doi.org/10.46586/tches.v2024.i2.329-358.

[CS20] G. Cassiers and F.-X. Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Transactions on Information Forensics and Security*, 15:2542–2555, 2020. https://doi.org/10.1109/TIFS.2020.2971153.

[CWE] CWE-226: Sensitive information in resource not removed before reuse. Common Weakness Enumeration (CWE). https://cwe.mitre.org/data/definitions/226.html.

[DGL17] D. Dinu, J. Großschädl, and Y. Le Corre. Efficient masking of ARX-based block ciphers using carry-save addition on Boolean shares. In *Information Security (ISC)*, LNCS 10599, pages 39–57. Springer-Verlag, 2017. https://doi.org/10.1007/978-3-319-69659-1_3.

[DKV26] E. Desmet, S. Kundu, and I. Verbauwhede. Masking out of order: Side-channel leaks from software-masked cryptography on out-of-order processors. Cryptology ePrint Archive, Paper 2026/123, 2026. https://eprint.iacr.org/2026/123.

[EM25] M. Escouteloup and V. Migliore. A hardware design methodology to prevent microarchitectural transition leakages. In *Constructive Approaches for Security Analysis and Design of Embedded Systems (CASCADE)*, LNCS 15952, pages 478–501. Springer-Verlag, 2025. https://doi.org/10.1007/978-3-032-01405-4_20.

[FIP19] Security requirements for cryptographic modules. National Institute of Standards and Technology (NIST) Federal Information Processing Standard (FIPS) 140-3, 2019. http://csrc.nist.gov.

[GJJR11] G. Goodwill, B. Jun, J. Jaffe, and P. Rohatgi. A testing methodology for side-channel resistance validation. NIST Non-Invasive Attack Testing Workshop, 2011. https://csrc.nist.gov/csrc/media/events/non-invasive-attack-testing-workshop/documents/08_goodwill.pdf.

[GMPP20] S. Gao, B. Marshall, D. Page, and T.H. Pham. FENL: an ISE to mitigate analogue micro-architectural leakage. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2020(2):73–98, 2020. https://doi.org/10.13154/tches.v2020.i2.73-98.

[GPM23] B. Gigerl, R. Primas, and S. Mangard. Secure context switching of masked software implementations. Cryptology ePrint Archive, Paper 2023/677, 2023. https://eprint.iacr.org/2023/677.

[Gut96] P. Gutmann. Secure deletion of data from magnetic and solid-state memory. In *USENIX Security Symposium*, pages 77–90, 1996. http://usenix.org/publications/library/proceedings/sec96/full_papers/gutmann/index.html.

[HOM06] C. Herbst, E. Oswald, and S. Mangard. An AES smart card implementation resistant to power analysis attacks. In *Applied Cryptography and Network Security (ACNS)*, LNCS 3989, pages 239–252. Springer-Verlag, 2006. https://doi.org/10.1007/11767480_16.

[HSH+09] J.A. Halderman, S.D. Schoen, N. Heninger, W. Clarkson, W. Paul, J.A. Calandrino, A.J. Feldman, J. Appelbaum, and E.W. Felten. Lest we remember: Cold-boot attacks on encryption keys. *Communications of the ACM*, 52(5):91–98, 2009. https://doi.org/10.1145/1506409.1506429.

[KJJ99] P.C. Kocher, J. Jaffe, and B. Jun. Differential power analysis. In *Advances in Cryptology (CRYPTO)*, LNCS 1666, pages 388–397. Springer-Verlag, 1999. https://doi.org/10.1007/3-540-48405-1_25.

[LHP20] T. Li, B. Hopkins, and S. Parameswaran. SIMF: Single-instruction multiple-flush mechanism for processor temporal isolation. *CoRR*, abs/2011.10249, 2020. https://arxiv.org/abs/2011.10249.

[MOP07] S. Mangard, E. Oswald, and T. Popp. *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer, 2007. https://doi.org/10.1007/978-0-387-38162-6.

[MPW22] B. Marshall, D. Page, and J. Webb. MIRACLE: MIcRo-ArChitectural Leakage Evaluation. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2022(1):175–220, 2022. https://doi.org/10.46586/tches.v2022.i1.175-220.

[OBG+24] S.A. Olmos, G. Barthe, R. Gonzalez, B. Grégoire, V. Laporte, J.-C. Lechenet, T. Oliveira, and P. Schwabe. High-assurance zeroization. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(2):375–397, 2024. https://doi.org/10.46586/tches.v2024.i1.375-397.

[PR13] E. Prouff and M. Rivain. Masking against side-channel attacks: A formal security proof. In *Advances in Cryptology (EUROCRYPT)*, LNCS 7881, pages 142–159. Springer-Verlag, 2013. https://doi.org/10.1007/978-3-642-38348-9_9.

[PV17] K. Papagiannopoulos and N. Veshchikov. Mind the gap: Towards secure 1st-order masking in software. In *Constructive Side-Channel Analysis and Secure Design (COSADE)*, LNCS 10348, pages 282–297. Springer-Verlag, 2017. https://doi.org/10.1007/978-3-319-64647-3_17.

[RV:19] The RISC-V instruction set manual. Technical Report Volume I: User-Level ISA (version 20190608-Base-Ratified), 2019. http://riscv.org/specifications.

[RV:23] RISC-V ABIs specification. Technical Report 1.1 (20231023), 2023. https://github.com/riscv/riscv-elf-psabi-doc.

[SACB24] I. Stork, V. Arora, Ł. Chmielewski, and I. Buhan. Towards explainable side-channel leakage: Unveiling the secrets of microarchitecture. Cryptology ePrint Archive, Paper 2024/1792, 2024. https://eprint.iacr.org/2024/1792.

[SCA18] L. Simon, D. Chisnall, and R. Anderson. What you get is what you C: Controlling side effects in mainstream C compilers. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 1–15, 2018. https://doi.org/10.1109/EuroSP.2018.00009.

[Sha79] A. Shamir. How to share a secret. *Communications of the ACM (CACM)*, 22(11):612–613, 1979. https://doi.org/10.1145/359168.359176.

[SSB+21] M.A. Shelton, N. Samwel, L. Batina, F. Regazzoni, M. Wagner, and Y. Yarom. Rosita: Towards automatic elimination of power-analysis leakage in ciphers. In *Network and Distributed System Security Symposium (NDSS)*, 2021. https://doi.org/10.14722/ndss.2021.23137.

[Sta19] F.-X. Standaert. How (not) to use Welch's T-test in side-channel security evaluations. In *Smart Card Research and Advanced Applications (CARDIS)*, LNCS 11389, pages 65–79. Springer-Verlag, 2019. https://doi.org/10.1007/978-3-030-15462-2_5.

[Wat16] A. Waterman. *Design of the RISC-V Instruction Set Architecture*. PhD thesis, University of California at Berkeley, 2016. https://people.eecs.berkeley.edu/~krste/papers/EECS-2016-1.pdf.

[ZBL+15] W. Zhang, Z. Bao, D. Lin, V. Rijmen, B. Yang, and I. Verbauwhede. RECTANGLE: a bit-slice lightweight block cipher suitable for multiple platforms. *Science China Information Sciences*, 58(12):1–15, 2015. https://doi.org/10.1007/s11432-015-5459-7.

[ZM24] J. Zeitschner and A. Moradi. PoMMES: Prevention of micro-architectural leakages in masked embedded software. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(3):342–376, 2024. https://doi.org/10.46586/tches.v2024.i3.342-376.